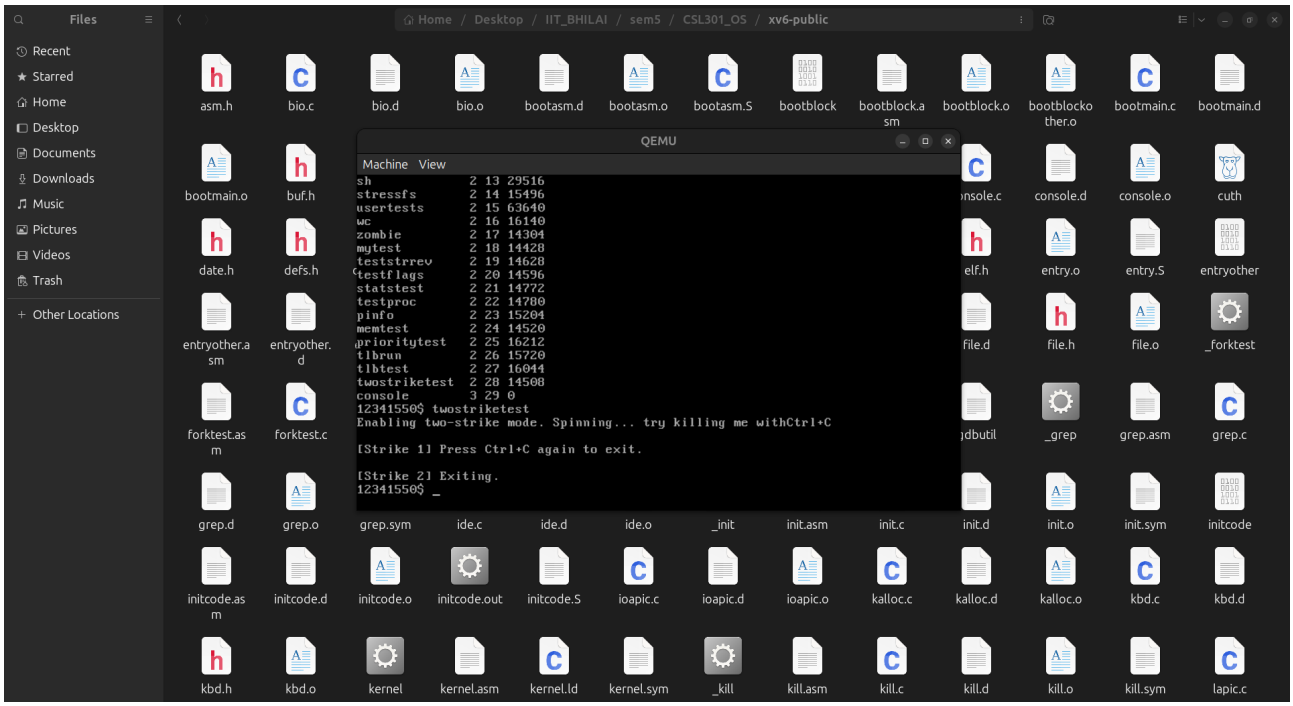
CLASS ASSIGNMENT – 11
ID : 12341550
DATE : 19-11-25

Q1

output:



Q2

code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <pwd.h>
```

```
#include <grp.h>
#include <unistd.h>
#include <time.h>
#include <limits.h>

#ifndef PATH_MAX
#define PATH_MAX 4096
#endif


void print_permissions(mode_t mode) {
    char type = '-';
    if (S_ISDIR(mode)) type = 'd';
    else if (S_ISLNK(mode)) type = 'l';
    else if (S_ISCHR(mode)) type = 'c';
    else if (S_ISBLK(mode)) type = 'b';
    else if (S_ISSOCK(mode)) type = 's';
    else if (S_ISFIFO(mode)) type = 'p';

    printf("%c", type);
    printf( (mode & S_IRUSR) ? "r" : "-");
    printf( (mode & S_IWUSR) ? "w" : "-");
    printf( (mode & S_IXUSR) ? "x" : "-");
    printf( (mode & S_IRGRP) ? "r" : "-");
    printf( (mode & S_IWGRP) ? "w" : "-");
    printf( (mode & S_IXGRP) ? "x" : "-");
    printf( (mode & S_IROTH) ? "r" : "-");
    printf( (mode & S_IWOTH) ? "w" : "-");
    printf( (mode & S_IXOTH) ? "x" : "-");
}


void print_file_details(const char *path, const char *name) {
    char fullpath[PATH_MAX];
```

```
    if (snprintf(fullpath, sizeof(fullpath), "%s/%s", path, name) >=
(int)sizeof(fullpath)) {
        fprintf(stderr, "Path too long: %s/%s\n", path, name);
        return;
    }

    struct stat sb;
    if (lstat(fullpath, &sb) == -1) {
        perror("stat");
        return;
    }

    print_permissions(sb.st_mode);

    struct passwd *pwd = getpwuid(sb.st_uid);
    struct group *grp = getgrgid(sb.st_gid);

    char timebuf[64];
    struct tm *tm_info = localtime(&sb.st_mtime);
    if (tm_info) {
        strftime(timebuf, sizeof(timebuf), "%b %e %H:%M",
tm_info);
    } else {
        strncpy(timebuf, "??? ?? ??:??", sizeof(timebuf));
        timebuf[sizeof(timebuf) - 1] = '\0';
    }

    printf(" %2ld %s %s %8lld %s %s\n",
           (long)sb.st_nlink,
           pwd ? pwd->pw_name : "?",
           grp ? grp->gr_name : "?",
           (long long)sb.st_size,
           timebuf,
           name);
```

```
}

int main(int argc, char *argv[]) {
    int long_format = 0;
    char *dirpath = NULL;


    if (argc == 1) {
        dirpath = ".";
    } else if (argc == 2) {
        if (strcmp(argv[1], "-l") == 0)
            dirpath = ".";
        else
            dirpath = argv[1];
    } else if (argc == 3 && strcmp(argv[1], "-l") == 0) {
        long_format = 1;
        dirpath = argv[2];
    } else {
        fprintf(stderr, "Usage: %s [-l] [directory]\n", argv[0]);
        exit(1);
    }

    if (argc >= 2 && strcmp(argv[1], "-l") == 0)
        long_format = 1;

    DIR *dir = opendir(dirpath);
    if (!dir) {
        perror("opendir");
        return 1;
    }

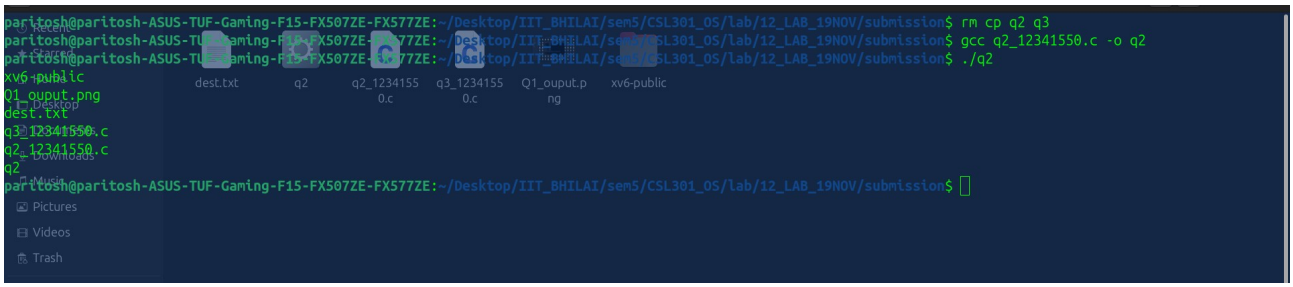
    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL) {
          if (strcmp(entry->d_name, ".") == 0 || strcmp(entry-
>d_name, "..") == 0)
```

```
            continue;

        if (long_format)
            print_file_details(dirpath, entry->d_name);
        else
            printf("%s\n", entry->d_name);
    }

    closedir(dir);
    return 0;
}
```

OUTPUT:



Q3

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>
#include <errno.h>

#define BUF_SIZE 4096
```

```
int copy_file(const char *src, const char *dest) {
    int src_fd, dest_fd;
    struct stat src_stat;
    char buffer[BUF_SIZE];
    ssize_t bytes_read, bytes_written;

    src_fd = open(src, O_RDONLY);
    if (src_fd < 0) { perror("open source"); return -1; }

    if (stat(src, &src_stat) < 0) {
        perror("stat");
        close(src_fd);
        return -1;
    }

    dest_fd = open(dest, O_WRONLY | O_CREAT | O_TRUNC,
src_stat.st_mode);
    if (dest_fd < 0) {
        perror("open destination");
        close(src_fd);
        return -1;
    }

    while ((bytes_read = read(src_fd, buffer, sizeof(buffer))) > 0) {
        ssize_t total_written = 0;

        while (total_written < bytes_read) {
            bytes_written = write(dest_fd, buffer + total_written,
bytes_read - total_written);
            if (bytes_written < 0) {
                perror("write");
                close(src_fd);
                close(dest_fd);
                unlink(dest);
```

```
            return -1;
        }
        total_written += bytes_written;
    }
}

if (bytes_read < 0) {
    perror("read");
    close(src_fd);
    close(dest_fd);
    unlink(dest);
    return -1;
}

close(src_fd);
close(dest_fd);
return 0;
}

int move_file(const char *src, const char *dest) {
    if (rename(src, dest) == 0) {
        return 0;
    }

    if (errno != EXDEV) {
        perror("rename");
        return -1;
    }

    if (copy_file(src, dest) < 0) {
        return -1;
    }

    if (unlink(src) < 0) {
        perror("unlink");
```

```
        unlink(dest);
        return -1;
    }

    return 0;
}

int main(int argc, char *argv[]) {
    const char *prog;
    const char *src;
    const char *dest;


    if (argc == 3) {
        // ln -s a.out cp
        // ln -s a.out mv
        // ./cp src dest  or ./mv src dest
        prog = strrchr(argv[0], '/');
        prog = (prog == NULL) ? argv[0] : prog + 1;
        src = argv[1];
        dest = argv[2];
    } else if (argc == 4) {
        // ./a.out cp src dest  or ./a.out mv src dest
        prog = argv[1];
        src = argv[2];
        dest = argv[3];
    } else {
        fprintf(stderr, "Usage:\n");
        fprintf(stderr, "  %s <source> <destination>\n", argv[0]);
        fprintf(stderr, "  %s <cp|mv> <source> <destination>\n",
argv[0]);
        exit(EXIT_FAILURE);
    }

    if (strcmp(prog, "cp") == 0) {
```

```
        if (copy_file(src, dest) == 0)
            printf("File copied successfully from '%s' to '%s'\n", src,
dest);
        else
            fprintf(stderr, "Copy failed.\n");
    }
    else if (strcmp(prog, "mv") == 0) {
        if (move_file(src, dest) == 0)
            printf("File moved successfully from '%s' to '%s'\n", src,
dest);
        else
            fprintf(stderr, "Move failed.\n");
    }
    else {
        fprintf(stderr, "Error: Invalid command name '%s'. Use cp or
mv.\n", prog);
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

OUPUT:



Q4

code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_INODES 8
#define MAX_BLOCKS 64
#define MAX_FILENAME 32
#define MAX_DESC 64
#define DIRECT_PTRS 4
#define MAX_DIR_ENTRIES 16

struct inode {
    int used;                  // 0 = free, 1 = allocated
    int size;                  // file size in bytes
    int type;                  // 0 = file, 1 = directory
    int creator_id;            // user ID who created file
    time_t created_at;         // creation timestamp
    char description[MAX_DESC];    // short file description
    int direct[DIRECT_PTRS];       // direct data block pointers
};

struct dir_entry {
    char name[MAX_FILENAME];
    int inum;
};

struct inode inode_table[MAX_INODES];
int data_block_bitmap[MAX_BLOCKS];   // 0 = free, 1 = used
struct dir_entry directory[MAX_DIR_ENTRIES];
int dir_count = 0;
```

```
int alloc_inode() {
    for (int i = 0; i < MAX_INODES; i++) {
        if (!inode_table[i].used) {
            inode_table[i].used = 1;
            inode_table[i].size = 0;
            inode_table[i].type = 0;
            inode_table[i].creator_id = 0;
            inode_table[i].created_at = time(NULL);
            strcpy(inode_table[i].description, "No description");
            for (int j = 0; j < DIRECT_PTRS; j++)
                inode_table[i].direct[j] = -1;
            return i;
        }
    }
    return -1;
}

void free_inode(int inum) {
    inode_table[inum].used = 0;
    inode_table[inum].size = 0;
    inode_table[inum].creator_id = 0;
    inode_table[inum].created_at = 0;
    strcpy(inode_table[inum].description, "");
    for (int j = 0; j < DIRECT_PTRS; j++)
        inode_table[inum].direct[j] = -1;
}

int alloc_block() {
    for (int i = 0; i < MAX_BLOCKS; i++) {
        if (data_block_bitmap[i] == 0) {
            data_block_bitmap[i] = 1;
            return i;
        }
```

```
    }
    return -1;
}

void free_block(int blocknum) {
    if (blocknum >= 0 && blocknum < MAX_BLOCKS)
        data_block_bitmap[blocknum] = 0;
}

int fs_create(const char *name, int creator_id, const char *desc) {
    for (int i = 0; i < dir_count; i++) {
        if (strcmp(directory[i].name, name) == 0) {
            printf("Error: File '%s' already exists\n", name);
            return -1;
        }
    }

    int inum = alloc_inode();
    if (inum < 0) {
        printf("Error: No free inode available\n");
        return -1;
    }
    if (dir_count >= MAX_DIR_ENTRIES) {
        printf("Error: Directory full\n");
        free_inode(inum);
        return -1;
    }

    inode_table[inum].creator_id = creator_id;
    inode_table[inum].created_at = time(NULL);
    inode_table[inum].size = 0;
    inode_table[inum].type = 0;
    strncpy(inode_table[inum].description, desc, MAX_DESC - 1);
    inode_table[inum].description[MAX_DESC - 1] = '\0';
```

```
    int block = alloc_block();
    if (block < 0) {
        printf("Error: No free data block available\n");
        free_inode(inum);
        return -1;
    }

    inode_table[inum].direct[0] = block;
    inode_table[inum].size = 0;

    strcpy(directory[dir_count].name, name);
    directory[dir_count].inum = inum;
    dir_count++;

    printf("File '%s' created (inode %d, block %d)\n", name, inum,
block);
    return inum;
}

int fs_delete(const char *name) {
    int found = -1;
    for (int i = 0; i < dir_count; i++) {
        if (strcmp(directory[i].name, name) == 0) {
            found = i;
            break;
        }
    }

    if (found < 0) {
        printf("Error: File '%s' not found\n", name);
        return -1;
    }

    int inum = directory[found].inum;
```

```
    for (int j = 0; j < DIRECT_PTRS; j++) {
        if (inode_table[inum].direct[j] != -1) {
            free_block(inode_table[inum].direct[j]);
            inode_table[inum].direct[j] = -1;
        }
    }

    free_inode(inum);

    for (int i = found; i < dir_count - 1; i++)
        directory[i] = directory[i + 1];
    dir_count--;

    printf("File '%s' deleted (inode %d freed)\n", name, inum);
    return 0;
}

void fs_show_state() {
    printf("\n--- File System State ---\n");
    printf("Directory entries (%d):\n", dir_count);
    for (int i = 0; i < dir_count; i++)
        printf("  %s → inode %d\n", directory[i].name,
directory[i].inum);

    printf("\nInode table:\n");
    for (int i = 0; i < MAX_INODES; i++) {
        if (inode_table[i].used) {
            printf("  inode %d: creator=%d, size=%d, desc='%s'\n",
                   i, inode_table[i].creator_id, inode_table[i].size,
inode_table[i].description);
            printf("            created: %s",
ctime(&inode_table[i].created_at));
            printf("            blocks: ");
            for (int j = 0; j < DIRECT_PTRS; j++)
```

```
            if (inode_table[i].direct[j] != -1)
                printf("%d ", inode_table[i].direct[j]);
            printf("\n");
        }
    }
    printf("-------------------------\n\n");
}


int main() {
    printf("File Creation and Deletion with Metadata Simulation\
n");

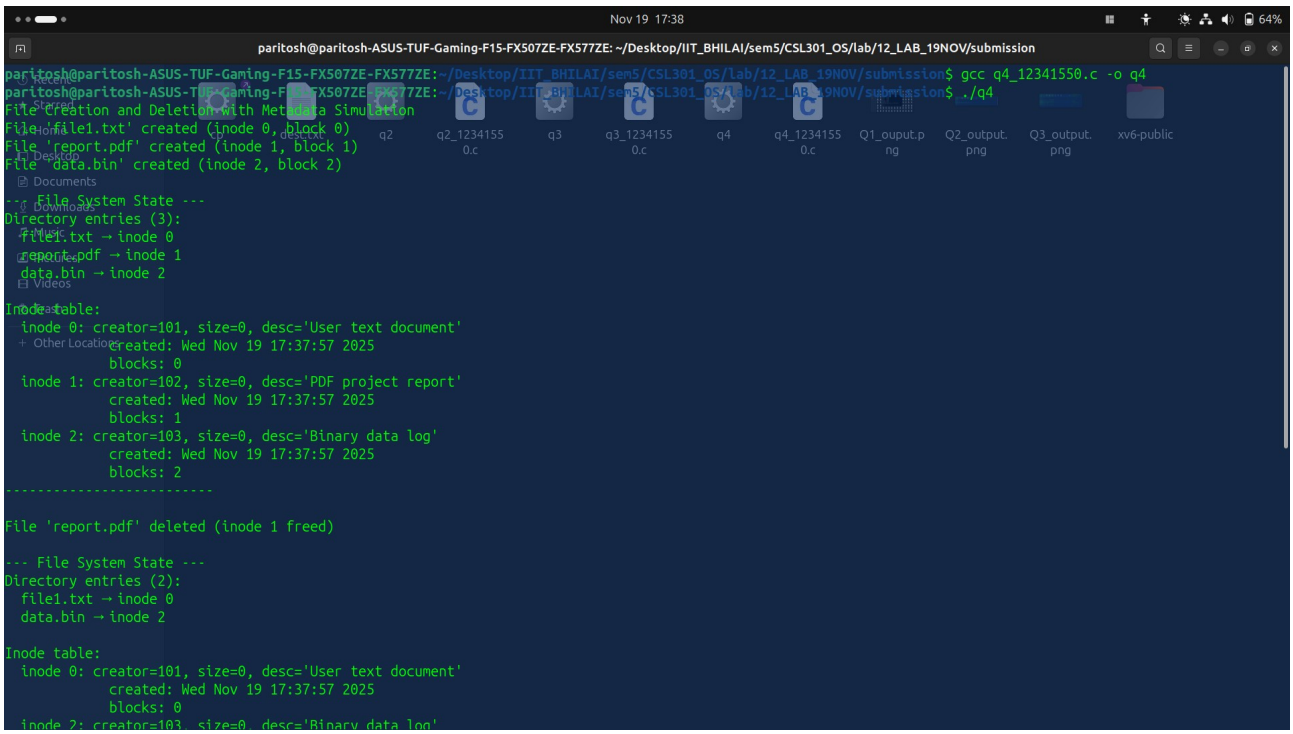
    fs_create("file1.txt", 101, "User text document");
    fs_create("report.pdf", 102, "PDF project report");
    fs_create("data.bin", 103, "Binary data log");
    fs_show_state();

    fs_delete("report.pdf");
    fs_show_state();

    fs_create("notes.txt", 104, "Important notes");
    fs_show_state();

    return 0;
}
```

OUPUT:


Nov 19 17:38
64%
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE: ~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/12_LAB_19NOV/submission
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE:~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/12_LAB_19NOV/submission$ gcc q4_12341550.c -o q4
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE:~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/12_LAB_19NOV/submission$ ./q4
File creation and Deletion with Metadata Simulation
File 'file1.txt' created (inode 0, block 0)
File 'report.pdf' created (inode 1, block 1)
File 'data.bin' created (inode 2, block 2)
--- File System State ---
Directory entries (3):
file1.txt → inode 0
report.pdf → inode 1
data.bin → inode 2
Inode table:
inode 0: creator=101, size=0, desc='User text document'
created: Wed Nov 19 17:37:57 2025
blocks: 0
inode 1: creator=102, size=0, desc='PDF project report'
created: Wed Nov 19 17:37:57 2025
blocks: 1
inode 2: creator=103, size=0, desc='Binary data log'
created: Wed Nov 19 17:37:57 2025
blocks: 2
File 'report.pdf' deleted (inode 1 freed)
--- File System State ---
Directory entries (2):
file1.txt → inode 0
data.bin → inode 2
Inode table:
inode 0: creator=101, size=0, desc='User text document'
created: Wed Nov 19 17:37:57 2025
blocks: 0
inode 2: creator=103, size=0, desc='Binary data log'
created: Wed Nov 19 17:37:57 2025
blocks: 2
File 'notes.txt' created (inode 1, block 1)
--- File System State ---
Directory entries (3):
file1.txt → inode 0
data.bin → inode 2
notes.txt → inode 1
Inode table:
inode 0: creator=101, size=0, desc='User text document'
created: Wed Nov 19 17:37:57 2025
blocks: 0
inode 1: creator=104, size=0, desc='Important notes'
created: Wed Nov 19 17:37:57 2025
blocks: 1
inode 2: creator=103, size=0, desc='Binary data log'
created: Wed Nov 19 17:37:57 2025
blocks: 2
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE:~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/12_LAB_19NOV/submission$